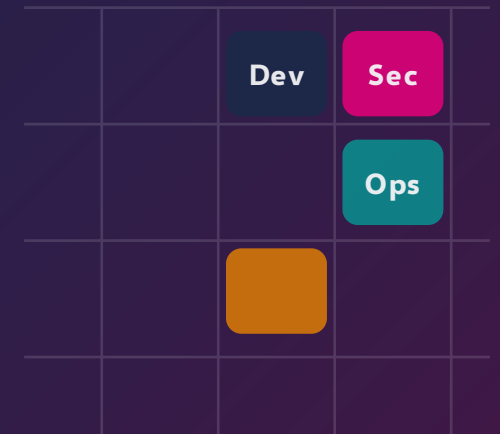


Techniques défensives et offensives des applications Web · Module 3

Introduction au DevSecOps

Intégrer la sécurité **tout au long** du cycle de développement, comme une responsabilité **partagée et automatisée** — pas comme une étape finale.



Objectifs pédagogiques

À la fin de ce module, vous saurez :

- Expliquer le « **shift left** » et pourquoi il fait gagner temps et argent.
- Définir le **DevSecOps** et ses grands **principes**.
- Situer la **collaboration** Dev · Ops · Sec et le rôle des **Security Champions**.
- Comprendre le poids de la **culture de sécurité** et du sans-blâme.
- Repérer les **outils** du pipeline (SAST, SCA, DAST, secrets) et les **frameworks** sécurisés.
- Écrire les premières **étapes de sécurité** dans un pipeline CI/CD.

L'idée centrale

Le DevSecOps **déplace la sécurité vers l'amont** et la rend **continue et automatisée**. Elle cesse d'être un contrôle en fin de course pour devenir un **réflexe d'équipe**, présent à chaque étape.

À retenir

« **Sec** » n'est pas une équipe qu'on appelle à la fin : c'est une **préoccupation partagée** par tout le monde, tout le temps.

■ Concept ■ Attention / risque ■ Bonne pratique

Sept angles pour comprendre le DevSecOps

- 1 Le « shift left »**
Placer la sécurité au plus tôt dans le cycle de développement
- 2 La méthodologie DevSecOps et ses principes**
Moindre privilège, défense en profondeur, sécurité par défaut
- 3 La collaboration Dev · Ops · Sec**
Responsabilités partagées, Security Champions
- 4 La culture de sécurité**
Sensibilisation, formation continue, retours sans blâme
- 5 Le panorama des outils**
Pipeline CI/CD, SAST · SCA · DAST · secrets, GitLab, Jenkins, SonarQube
- 6 Les frameworks sécurisés**
Spring Security, Express.js avec Helmet
- 7 La sécurité dans l'intégration continue**
Première approche du SAST et du DAST

01

La sécurité dans le cycle de développement : le « shift left »

Plus une faille est trouvée tôt, moins elle coûte cher à corriger.

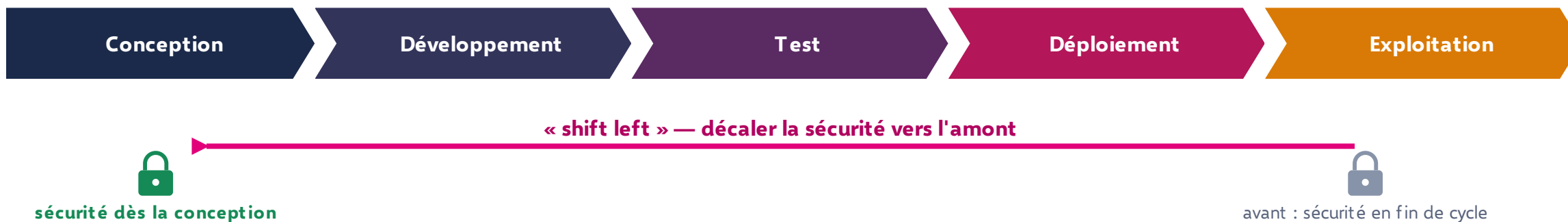
Dans cette partie

- Le cycle de vie logiciel (SDLC)
- Le coût de correction par phase
- Ce qu'est le « shift left »
- Avant / après en pratique

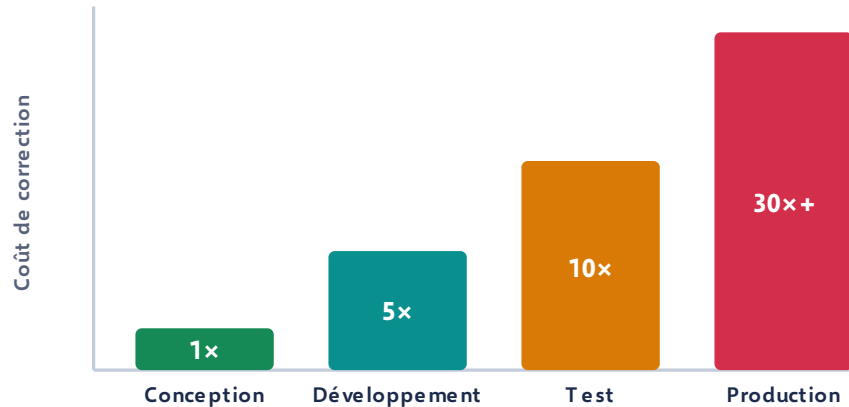
Où placer la sécurité ?

Définition — SDLC

Le **SDLC** (Software Development Life Cycle) enchaîne les étapes d'un projet. Longtemps, la sécurité n'intervenait qu'**à la toute fin** (audit avant mise en production) — trop tard pour corriger sans douleur. Le « **shift left** » la déplace vers l'amont.



Le coût de correction explose avec le temps



Le message

Corriger une faille en **conception** coûte une fraction de ce qu'elle coûtera en **production** (correctif d'urgence, incident, image de marque). D'où l'intérêt de « **pousser à gauche** ».

Confirmé par les chiffres

Les programmes DevSecOps matures réduisent les vulnérabilités en production de **60 à 70 %** et interceptent 40 à 50 % des problèmes dès la phase de code, là où corriger ne coûte presque rien.

Ordres de grandeur indicatifs (modèle historique) ; réductions : synthèses DevSecOps 2026.

Qu'est-ce que le « shift left » ?

Définition

Le « **shift left** » (décaler vers la gauche) consiste à **déplacer les activités de sécurité vers l'amont** du cycle : dès la conception et l'écriture du code, et non juste avant la mise en production.

Concrètement :

- Penser aux menaces **dès la conception** (threat modeling — vu au module 2).
- Analyser le code **à chaque commit**, automatiquement.
- Donner un **retour rapide** au développeur, dans son outillage.

Exemple

Au lieu de découvrir une injection SQL lors d'un audit final, un outil la signale **dans l'IDE** ou dès l'intégration continue, **quelques minutes** après son écriture — quand la corriger ne coûte presque rien.

Attention

« Shift left » ne veut pas dire **supprimer** les contrôles finaux (audit, test d'intrusion). On **ajoute** de la sécurité en amont, on ne retire rien en aval. On parle même de « shift left **et** right » : sécuriser aussi en production (surveillance).

Avant / après le « shift left »

🕒 Approche traditionnelle

- **Quand** : audit de sécurité en fin de projet
- **Qui** : une équipe sécurité séparée
- **Comment** : contrôles manuels, ponctuels
- **Coût d'une faille** : élevé (correction tardive)
- **Effet** : la sécurité ralentit la livraison

✅ Approche « shift left »

- **Quand** : sécurité à chaque étape, en continu
- **Qui** : Dev, Ops et Sec ensemble
- **Comment** : analyses automatisées dans le pipeline
- **Coût d'une faille** : faible (correction immédiate)
- **Effet** : la sécurité suit le rythme des livraisons

La bascule

On ne rajoute pas une étape « sécurité » : on **diffuse** la sécurité dans toutes les étapes existantes, portée par l'automatisation. C'est tout l'objet du DevSecOps.

02

La méthodologie DevSecOps et ses principes

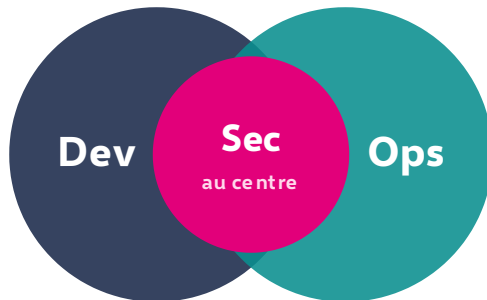
Du DevOps au DevSecOps : la sécurité comme responsabilité de tous.

Dans cette partie

- De DevOps à DevSecOps
- « La sécurité est l'affaire de tous »
- Moindre privilège
- Défense en profondeur
- Sécurité par défaut
- Trois réflexes structurants

De DevOps à DevSecOps

automatisation · mesure · amélioration continue



DevOps a rapproché le **développement** (Dev) et l'**exploitation** (Ops) pour livrer **plus vite et plus souvent**, en automatisant la chaîne.

Définition — DevSecOps

Le **DevSecOps** ajoute la **sécurité (Sec)** à cette collaboration : elle est intégrée **dès le départ et en continu**, et non ajoutée après coup.

La nuance

Le « Sec » n'est pas une troisième équipe posée entre les deux autres : c'est une **préoccupation commune** qui irrigue tout le cycle.

« La sécurité est l'affaire de tous »

Le DevSecOps repose sur trois piliers :

- **Automatiser** la sécurité dans le pipeline (tests, scans, portes de qualité).
- **Collaborer** : plus de silo entre Dev, Ops et Sec.
- **Améliorer en continu** : mesurer, apprendre, corriger.

Mesurer, aussi

Un programme mature suit des **métriques** : délai de correction, dette de sécurité (vulnérabilités critiques ouvertes), taux de re-test réussi. « Ce qui se mesure s'améliore. »

Exemple d'automatisation

À chaque `git push`, le pipeline lance automatiquement l'analyse du code, le scan des dépendances et des tests de sécurité. Un résultat **bloquant** empêche la livraison — sans intervention humaine.

Le piège 2026

Le code **généré par IA** accélère la production mais crée des angles morts : ~95 % des organisations utilisent l'IA pour développer, mais une minorité évalue sérieusement la sécurité du code produit. D'où l'importance des portes automatiques.

Sources : Black Duck / GitLab DevSecOps 2025-2026.

Le moindre privilège

Définition

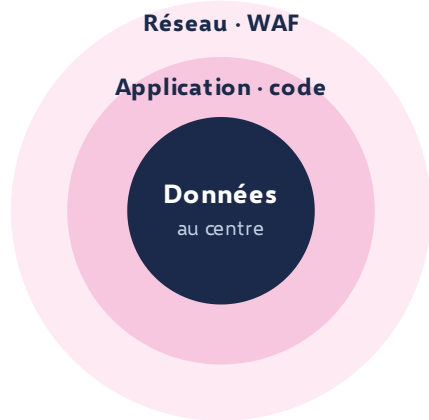
Le **moindre privilège** (least privilege) : chaque utilisateur, service ou composant ne reçoit **que les droits strictement nécessaires** à sa tâche, et rien de plus.

Objectif : **limiter le rayon d'explosion** (blast radius) si un compte ou un service est compromis. C'est aussi un pilier de l'approche **Zero Trust** : ne jamais faire confiance par défaut, vérifier à chaque accès.

Exemples

- Un service qui ne fait que **lire** une base n'a pas les droits d'**écriture** ni de **suppression**.
- Un conteneur ne tourne pas en `root`.
- Un jeton d'API n'accède qu'aux ressources qu'il utilise, et expire.

La défense en profondeur



chaque couche protège si la précédente cède

Définition

La **défense en profondeur** empile **plusieurs couches de protection indépendantes** : si l'une cède, les suivantes tiennent encore.

On ne mise jamais sur une **seule** barrière : validation des entrées **ET** requêtes paramétrées **ET** WAF **ET** moindre privilège en base...

Analogie

Un château : douves, remparts, portes, gardes. L'attaquant doit **tout** franchir, pas une seule barrière.

La sécurité par défaut

Définition

La **sécurité par défaut** (secure by default) : la configuration **livrée d'origine** est déjà la plus sûre. L'utilisateur doit faire un choix **explicite** pour l'affaiblir, jamais l'inverse.

Elle va de pair avec le **secure by design** : penser la sécurité **dès la conception** — et rejoint le « privacy by design / by default » du RGPD vu au module 2.

Exemples

- Pas de mot de passe par défaut (`admin/admin`).
- Chiffrement (HTTPS) activé d'emblée.
- Ports et services inutiles **fermés** par défaut.
- Permissions **restrictives** au départ, élargies au besoin.

Trois réflexes structurants



Moindre privilège

Juste les droits nécessaires, rien de plus



Défense en profondeur

Plusieurs couches indépendantes



Sécurité par défaut

Le réglage sûr est celui d'origine

À garder en tête

Ces principes ne sont pas propres au DevSecOps : ce sont des **fondamentaux** de la sécurité, que le DevSecOps **outille et automatise** à grande échelle, à chaque livraison.

03

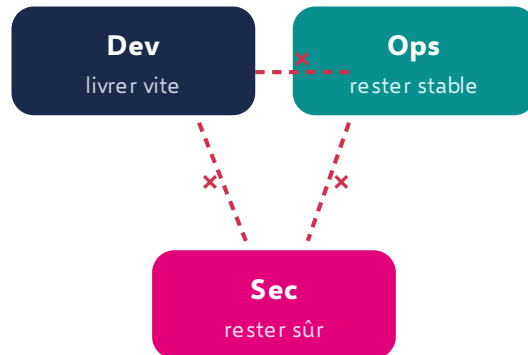
La collaboration entre les équipes

Casser les silos entre développement, exploitation et sécurité.

Dans cette partie

- Trois équipes, trois objectifs
- Un objectif commun
- Les responsabilités partagées
- Les Security Champions

Trois équipes, trois objectifs, des silos



Traditionnellement, chaque équipe poursuit un but qui peut **entrer en tension** avec les autres :

- **Dev** : livrer des fonctionnalités **vite**.
- **Ops** : garder le système **stable**.
- **Sec** : garder le système **sûr**.

Chacun dans son **silo**, la sécurité arrive en dernier et se vit comme un **frein**.

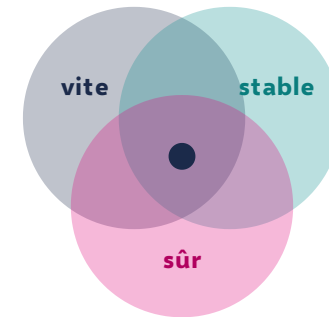
Un objectif commun : vite, stable ET sûr

La bascule DevSecOps

On remplace des objectifs qui s'opposent par un **objectif commun** : livrer **vite, stable ET sûr**. La sécurité devient un **critère de qualité partagé**, au même titre que la performance ou la fiabilité.

Ce que disent les données

Les équipes DevSecOps matures déploient jusqu'à **2 fois plus souvent** et réduisent le délai de livraison de ~66 % — tout en **abaissant** le risque. Vitesse et sécurité ne s'opposent plus : elles se renforcent.



Chiffres : synthèses DevSecOps / rapports DORA 2025-2026.

La sécurité, un travail d'équipe

Rôle	Contribue à la sécurité en...
Développeur	Écrivant du code sûr, validant les entrées, corrigeant les alertes des outils
Ops / SRE	Durcissant les serveurs, gérant les secrets, surveillant, appliquant les correctifs
Équipe sécurité	Outillant, formant, définissant les exigences et les tests, accompagnant
Direction	Donnant les moyens, portant la culture, arbitrant les risques

Le modèle vaut aussi pour le cloud

Dans la **responsabilité partagée** cloud, le fournisseur sécurise l'infrastructure, mais **vous** restez responsable de vos données, de vos accès et de vos configurations. Une mauvaise config reste de votre côté.

Les Security Champions

Définition

Un **Security Champion** est un **développeur volontaire** de l'équipe, formé à la sécurité, qui en devient le **relais au quotidien** — sans être un expert à plein temps.

Il fait le **pont** entre l'équipe de développement et l'équipe sécurité : celle-ci ne peut pas être partout, le champion diffuse la culture au plus près du code.

Son rôle concret

- Diffuser les **bonnes pratiques** dans son équipe
- Repérer les sujets à risque **tôt**
- Aider aux **revues de code** orientées sécurité
- Faire remonter les besoins vers les experts

04

La culture de sécurité

Le facteur humain est décisif : aucun outil ne remplace les bons réflexes.

Dans cette partie

- Pourquoi les outils ne suffisent pas
- Sensibilisation et formation continue
- Les retours d'expérience sans blâme

Les outils ne suffisent pas

On peut avoir les meilleurs outils : si les personnes ne sont pas **sensibilisées**, la sécurité échoue.

- Un clic sur un e-mail piégé contourne les défenses techniques.
- Un **secret** laissé dans le code annule le chiffrement.
- Une alerte **ignorée par lassitude** laisse passer une faille.

Rappel du module 1

L'**élément humain** est impliqué dans la majorité des violations. La technique seule ne protège pas d'un geste humain.

Définition

La **culture de sécurité** est l'ensemble des **réflexes, valeurs et comportements** partagés qui font de la sécurité une préoccupation **naturelle et quotidienne**, à tous les niveaux.

La fatigue d'alerte

Trop d'alertes (souvent des faux positifs) finit par être ignorée. Régler la sensibilité des outils et prioriser fait partie de la culture, pas seulement de la technique.

Sensibilisation et formation continue



Formation continue

Se former régulièrement,
pas une fois pour toutes



Simulations

Faux phishing, exercices,
pour ancrer les réflexes



Sensibilisation

Rappels, communication,
exemples concrets



Valorisation

Reconnaître les bons
comportements



Ressources

Guides, Cheat Sheets
OWASP, standards
accessibles



Répétition

Rappels réguliers plutôt
qu'un gros bloc annuel

Exemple

Une campagne de **faux phishing** interne mesure le taux de clics, cible la formation là où c'est utile, et fait **progresser** la vigilance dans la durée — à condition d'être menée sans humilier les personnes qui cliquent.

Les retours d'expérience sans blâme

Définition

Un **retour d'expérience sans blâme** (blameless post-mortem) analyse un incident pour comprendre **ce qui a échoué dans le système** — pas pour désigner un coupable.

On cherche la **cause profonde** (processus, outil, manque de garde-fou) plutôt que « qui a fait l'erreur ».

Pourquoi « sans blâme » ?

Si les gens ont peur d'être sanctionnés, ils **cachent** les incidents. La transparence, elle, permet de **corriger durablement** et d'éviter la récurrence.

Le bon réflexe

« **Comment le système a-t-il permis cette erreur ?** » plutôt que « qui est fautif ? ». Une erreur humaine possible est d'abord un garde-fou manquant.

05

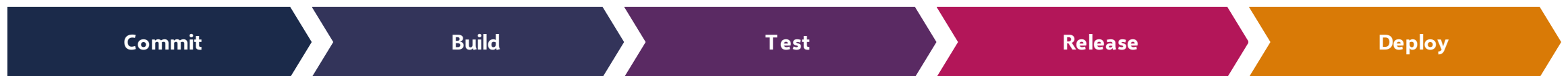
Panorama des outils

La chaîne d'intégration et de déploiement continu, socle de l'automatisation.

Dans cette partie

- La chaîne CI/CD
- Un pipeline DevSecOps réel
- Les mêmes contrôles ailleurs
- La boîte à outils par catégorie
- GitLab, Jenkins, SonarQube

La chaîne CI/CD



Définition — CI/CD

CI (intégration continue) : intégrer et tester le code **en continu** à chaque modification. **CD** (livraison / déploiement continu) : livrer et déployer de façon **automatisée**.

Le lien avec la sécurité

Le pipeline CI/CD est l'endroit idéal pour **insérer les contrôles de sécurité automatiques**. C'est le cœur du DevSecOps : chaque livraison passe par les mêmes garde-fous.

Le pipeline est une cible

Le pipeline lui-même doit être durci : secrets protégés, dépendances de build vérifiées, accès restreint. Un pipeline compromis = tout ce qu'il produit est compromis.

Un pipeline DevSecOps réel : `.gitlab-ci.yml`

Ajouter quatre contrôles de sécurité au pipeline tient en quelques lignes :

```
stages: [build, test, security, deploy]

build:
  stage: build
  image: maven:3.9-eclipse-temurin-21
  script: mvn -B clean package

sast:
  # analyse du code (SAST)
  stage: security
  script: semgrep ci --config=p/owasp-top-ten

deps:
  # dépendances + image (SCA)
  stage: security
  script: trivy fs --exit-code 1 --severity HIGH,CRITICAL .

secrets:
  # clés / mots de passe oubliés
  stage: security
  script: gitleaks detect --source . --no-git

dast:
  # application qui tourne (DAST)
  stage: security
  script: zap-baseline.py -t https://staging.monapp.fr
```

Chaque job de sécurité **échoue et bloque la livraison** si un problème grave est détecté — la sécurité devient automatique.

Les mêmes contrôles en GitHub Actions

```
name: ci-security
on: [push]

jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: SAST (Semgrep)
        run: semgrep ci --config=p/owasp-top-ten

      - name: Dépendances (Trivy)
        uses: aquasecurity/trivy-action@master
        with: { scan-type: fs, severity: HIGH,CRITICAL }

      - name: Secrets (Gitleaks)
        uses: gitleaks/gitleaks-action@v2
```

Même logique, autre syntaxe

GitLab CI, GitHub Actions, Jenkins... changent la **forme**, pas le **principe** : des **étapes automatiques** de sécurité dans le pipeline.

Portable

Comme les scanners sont des exécutables ou des images Docker, on migre d'un CI à l'autre sans réécrire la logique de sécurité.

Les outils concrets, par catégorie

SAST — CODE

Semgrep · SonarQube · CodeQL

Analyse statique du code source

SCA — DÉPENDANCES

Trivy · Dependency-Check · Gripe

Bibliothèques tierces vulnérables

SECRETS

Gitleaks · TruffleHog · detect-secrets

Clés / mots de passe oubliés dans le code

CONTENEURS

Trivy · Gripe · Docker Scout

Vulnérabilités des images Docker

IAC — INFRA AS CODE

Checkov · tfsec · KICS

Terraform, Kubernetes, config cloud

DAST — EXÉCUTION

OWASP ZAP · Nikto · Nuclei

Test de l'application en fonctionnement

Bon à savoir

La plupart sont **open source** et se lancent en **une commande** ou une image Docker — donc intégrables dans n'importe quel pipeline. Un outil « tout-en-un » (Trivy) couvre souvent plusieurs catégories.

Sécuriser la chaîne d'approvisionnement : SBOM

Définition — SBOM

Un **SBOM** (Software Bill of Materials, nomenclature logicielle) est l'**inventaire complet** des composants et dépendances d'une application — l'équivalent de la liste des ingrédients.

Il répond en quelques secondes à « **suis-je affecté par cette CVE ?** » : au lieu de fouiller, on interroge l'inventaire. Formats standard : **CycloneDX**, **SPDX**. Le cadre **SLSA** y ajoute la **provenance** vérifiable des artefacts (qui a construit quoi, comment).

87 %

des organisations exécutent au moins un service avec une vulnérabilité connue exploitable (Datadog 2026)

×2,5

de paquets malveillants publiés en 2025 vs 2024 dans les dépôts open source (Sonatype)

Générer un SBOM en une commande

```
trivy fs --format cyclonedx -o sbom.json .
```

 — puis on le rescane à chaque nouvelle CVE, sans reconstruire l'application.

GitLab CI/CD

Définition

GitLab CI/CD est la chaîne d'intégration/déploiement **intégrée à la plateforme GitLab**. Les étapes du pipeline se décrivent dans un fichier `.gitlab-ci.yml` versionné avec le code.

Chaque `push` ou *merge request* déclenche automatiquement le pipeline.

Atout pour le DevSecOps

GitLab propose des **étapes de sécurité prêtes à l'emploi** (analyse du code, scan des dépendances, scan de conteneurs, détection de secrets) à activer en quelques lignes, avec un **tableau de bord** des vulnérabilités intégré.

Jenkins

Définition

Jenkins est un serveur d'automatisation **open source** historique, très **extensible** (large écosystème de plugins). Les pipelines se décrivent dans un **Jenkinsfile** versionné avec le code.

Très flexible, mais demande **plus de configuration** qu'une solution intégrée.

```
pipeline {  
  agent any  
  stages {  
    stage('Build') { steps { sh 'mvn -B package' } }  
    stage('SAST') { steps { sh 'semgrep ci' } }  
    stage('SCA') { steps { sh 'trivy fs .' } }  
    stage('DAST') { steps { sh 'zap-baseline.py -t $URL' } }  
  }  
}
```

Les mêmes scanners, orchestrés par Jenkins.

SonarQube

Définition

SonarQube est une plateforme d'**analyse statique du code** : elle mesure la **qualité** et détecte bugs, code peu maintenable et **vulnérabilités** (c'est un outil de **SAST**).

```
# lancer l'analyse (projet Maven)
mvn sonar:sonar \
  -Dsonar.host.url=http://sonarqube:9000 \
  -Dsonar.token=$SONAR_TOKEN
```

Quality Gate

Une **porte de qualité** : si le code ne respecte pas les critères (trop de vulnérabilités, couverture de tests insuffisante...), le pipeline **échoue** et la livraison est **bloquée**. La sécurité devient une condition automatique.

Bonne pratique

Régler la Quality Gate sur le **code neuf** (« clean as you code ») : on n'exige pas de tout corriger d'un coup, mais que **chaque nouveau commit** soit propre.

06

Les frameworks sécurisés

S'appuyer sur des briques éprouvées plutôt que réinventer la sécurité.

Dans cette partie

- Ne pas réinventer la sécurité
- Spring Security (Java)
- Express.js avec Helmet (Node.js)

Ne pas réinventer la sécurité

Écrire soi-même son authentification, son chiffrement ou ses entêtes de sécurité est **risqué** : c'est difficile, et la moindre erreur crée une faille exploitable.

Bonne pratique

S'appuyer sur des **frameworks et bibliothèques éprouvés**, maintenus et audités par une large communauté — ils encapsulent les bonnes pratiques et sont corrigés en continu.

« Don't roll your own crypto »

La règle d'or : ne jamais réimplémenter soi-même un mécanisme de sécurité critique (cryptographie, gestion de session, hachage de mots de passe) quand une solution reconnue existe.

Le revers

Une dépendance est aussi une **surface d'attaque** (cf. SBOM). Choisir des briques **maintenues**, les **tenir à jour** et les **scanner** (SCA) fait partie du contrat.

Spring Security

Définition

Spring Security est le module de sécurité du framework **Spring** (Java) : authentification, autorisation et protections prêtes à l'emploi.

```
@Bean
SecurityFilterChain security(HttpSecurity http) {
    http
        .authorizeHttpRequests(a -> a
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .csrf(Customizer.withDefaults()) // anti-CSRF
        .formLogin(Customizer.withDefaults());
    return http.build();
}
```

Ce qu'il apporte

- Authentification (formulaire, OAuth2, LDAP...)
- Autorisation par **rôles** et par règles
- Protection **CSRF**, en-têtes de sécurité
- Sessions et mots de passe (**BCrypt**) sécurisés

Express.js avec Helmet

Définition

Express.js = framework Web **minimaliste** pour Node.js. **Helmet** = **middleware** qui pose automatiquement des **en-têtes HTTP de sécurité**.

```
const express = require('express');
const helmet = require('helmet');
const app = express();

app.use(helmet()); // en-têtes sécurisés
app.use(helmet.hsts({ maxAge: 31536000 }));
app.disable('x-powered-by'); // masquer la techno
```

Ce que règle Helmet

- **CSP** (Content Security Policy) contre le XSS
- **HSTS** pour forcer le HTTPS
- **X-Frame-Options** contre le clickjacking
- Masque les en-têtes qui révèlent la techno

07

La sécurité dans l'intégration continue

Première approche du SAST et du DAST — approfondie plus loin dans la formation.

Dans cette partie

- Où agir dans le pipeline
- Le SAST (analyse statique)
- Le DAST (analyse dynamique)
- SAST vs DAST et compléments
- Lancer les scanners

Un contrôle de sécurité à chaque étape



Idée

Chaque type d'analyse se place là où il est pertinent : le **code source** pour le SAST, les **dépendances** pour le SCA, la **config** pour l'IaC, l'**application qui tourne** pour le DAST, et la **veille** continue en production.

Le SAST

Définition — SAST

Le **SAST** (Static Application Security Testing) analyse le **code source** (ou compilé) **sans l'exécuter**, pour détecter des vulnérabilités : injections, secrets en dur, mauvaises pratiques.

Boîte blanche : il « voit » le code, et pointe la **ligne** fautive.

Apports et limites

- + Très **tôt** (dès le commit), pointe la ligne fautive
- + Couvre tout le code, même les chemins rares
- – Génère des **faux positifs** à trier
- – Ne voit pas les failles d'**exécution** / de configuration

Le DAST

Définition — DAST

Le **DAST** (Dynamic Application Security Testing) teste l'application **en cours d'exécution**, de l'**extérieur**, en envoyant de vraies requêtes — comme le ferait un attaquant.

Boîte noire : il ne voit pas le code, seulement le **comportement**.

Apports et limites

- + Trouve des failles **réelles** et d'**exécution** (config, en-têtes)
- + Peu de faux positifs sur ce qu'il confirme
- – Plus **tard** (appli déployée), ne pointe pas la ligne
- – Ne teste que ce qu'il **atteint** réellement

SAST, DAST et compléments

	SAST	DAST
Objet analysé	Le code source	L'application qui tourne
Point de vue	Boîte blanche	Boîte noire (comme un attaquant)
Quand	Très tôt (commit, build)	Plus tard (appli déployée)
Trouve surtout	Faibles de code	Faibles d'exécution et de configuration
Faux positifs	Plutôt nombreux	Plutôt rares (sur failles confirmées)

Complémentaires, pas concurrents

On les **combine**, avec le **SCA** (analyse des dépendances) et la **détection de secrets**. C'est la défense en profondeur du pipeline. Approche approfondie plus loin dans la formation.

Lancer les scanners, concrètement

```
# SAST – analyser le code source
semgrep scan --config=p/owasp-top-ten src/

# SCA + conteneur – dépendances et image Docker vulnérables
trivy fs --severity HIGH,CRITICAL .
trivy image monapp:latest

# Secrets – clés / mots de passe oubliés dans le dépôt
gitleaks detect --source .

# DAST – tester l'application qui tourne (OWASP ZAP)
docker run ghcr.io/zaproxy/zaproxy zap-baseline.py \
  -t https://staging.monapp.fr
```

Le point commun

Chaque outil s'exécute en **une commande** (ou une image Docker) et renvoie un **code d'échec** en cas de problème grave — c'est ce qui permet de **bloquer automatiquement** le pipeline.

Les termes du module en un coup d'œil

DevOps — collaboration Développement + Exploitation

Shift left — déplacer la sécurité vers l'amont

Pipeline — chaîne d'étapes automatisées

Ops / SRE — exploitation / Site Reliability Engineering

SAST — analyse statique du code source

IaC — Infrastructure as Code (Terraform, K8s...)

Secret — clé, mot de passe, jeton à ne pas exposer

CSP — Content Security Policy (en-tête anti-XSS)

Middleware — composant intercalé dans le traitement des requêtes

DevSecOps — DevOps intégrant la Sécurité en continu

Shift right — sécuriser aussi en production (surveillance)

Quality Gate — seuil de qualité bloquant

Zero Trust — ne jamais faire confiance par défaut

DAST — test de l'application en exécution

SBOM — nomenclature logicielle (inventaire des composants)

CSRF — Cross-Site Request Forgery

HSTS — force le HTTPS

BCrypt — algorithme de hachage de mots de passe

SDLC — cycle de vie du développement logiciel

CI/CD — intégration / livraison-déploiement continu

Security Champion — développeur relais de la sécurité

Moindre privilège — juste les droits nécessaires

SCA — analyse des dépendances tierces

SLSA — cadre de provenance et d'intégrité des artefacts

XSS — Cross-Site Scripting

WAF — pare-feu applicatif Web

Post-mortem sans blâme — analyse d'incident sans désigner de coupable

Ce qu'il faut retenir de ce module

- 1 Le « **shift left** » place la sécurité au plus tôt : plus une faille est trouvée tôt, moins elle coûte cher.
- 2 Le **DevSecOps** rend la sécurité **continue, automatisée et partagée**. Trois principes : moindre privilège, défense en profondeur, sécurité par défaut.
- 3 **Dev, Ops et Sec** visent un objectif commun : vite, stable ET sûr. **Security Champions** et **culture sans blâme** font la différence.
- 4 Le **pipeline CI/CD** accueille les contrôles automatiques : **SAST, SCA, secrets, IaC, DAST**, et le **SBOM** pour la chaîne d'approvisionnement.
- 5 On **réutilise** des frameworks éprouvés (Spring Security, Helmet) plutôt que réinventer la sécurité — tout en surveillant ses dépendances.

Et ensuite ?

On entre dans le concret des vulnérabilités et des protections :

Technologies de sécurisation

OWASP Top 10:2025 en pratique

SAST / DAST / SCA approfondis

L'idée à emporter

La sécurité n'est plus une **étape finale** mais une **propriété continue** du cycle de développement — outillée, mesurée et portée par toute l'équipe.

Sources et références

Cadres et pratiques

- **OWASP** — DevSecOps Guideline, SAMM, Cheat Sheets · owasp.org
- **OWASP ZAP** — DAST open source · zapproxy.org
- **SLSA** — provenance des artefacts · slsa.dev
- **CycloneDX / SPDX** — formats de SBOM

Outils cités

- **Semgrep, SonarQube, CodeQL** — SAST
- **Trivy, Gripe, Dependency-Check** — SCA
- **Gitleaks, TruffleHog** — secrets · **Checkov** — IaC

Chiffres cités

- **Datadog** — State of DevSecOps 2026
- **Sonatype** — State of the Software Supply Chain 2025
- **GitLab / Black Duck** — DevSecOps & IA 2025-2026
- **IBM** — Cost of a Data Breach 2025

Points à actualiser

Les rapports (Datadog, Sonatype, GitLab, IBM) sont **annuels** ; les versions d'outils et d'images Docker évoluent vite. Vérifier chiffres et versions avant chaque session.

Fin du module 3

Des questions ?

La sécurité n'est plus une **étape finale** mais une **démarche continue et partagée**.
Place aux technologies de sécurisation et aux vulnérabilités concrètes de l'OWASP Top 10.